Fed-DCSA Experiment Context

What this project is

A Python numerical simulation of the **Fed-DCSA** algorithm (a federated distributed optimization method) applied to a multi-drone sector-coverage problem. This is for a robotics journal paper. There are no real drones, no ROS, no physics engine — just optimization loops that produce convergence and feasibility plots to validate the theory (Theorem 1) in the paper.

The optimization problem

A fleet of n identical drones surveys d geographic sectors under a shared per-round energy budget b (watt-hours).

Decision variable per drone: A coverage-time vector $x_i = (x_{i1}, ..., x_{id})$ where $x_{ij} \geq 0$ is the minutes drone i allocates to sector j this round. Bounded by a local feasible set: all components non-negative and $\sum_j x_{ij} \leq T_{max}$ (round duration in minutes).

Local cost: $f_i(x_i) = \sum_j \alpha_{ij} * (x_{ij} - \theta_{ij})^2$ — weighted squared deviation from coverage demands. θ_{ij} is how many minutes drone i should ideally spend in sector j; α_{ij} is the priority weight.

Coupled constraint: $G(x) = \sum_i \sum_j c_{ij} * x_{ij} - b \leq 0$ — the fleet's total energy usage cannot exceed the budget. c_{ij} is the energy cost rate (Wh/min) for drone i in sector j. To make this separable as $G = \sum_i g_i$, define $g_i(x_i) = \sum_j c_{ij} * x_{ij} - b/n$.

Gradients are closed-form and cheap:

- $\nabla f_i(x_i) = (2 * \alpha_{ij} * (x_{ij} - \theta_{ij}))$ for each component j
- $\nabla g_i(x_i) = (c_{ij})$ — a constant vector

Projection onto X_i is: clamp negatives to 0, then if the sum exceeds T_{max}, project onto the capped simplex (standard O(d log d) sort-and-threshold).

The algorithm (Fed-DCSA)

A two-loop server–client protocol with K outer communication rounds and T inner local gradient steps per round.

Communication phase (once per round): Each drone sends a single scalar $g_i(x_i)$ to the server. The server sums them to get $G_{tilde} \approx G(x)$, computes a drift margin r_{drift} (accounts for how much the constraint can move during T inner steps), and broadcasts a binary **gate** decision to all drones.

Gate: `b_k = 1` if `G_tilde ≤ eta_k - r_k - r_drift`, else `b_k = 0`. In the deterministic case, `r_k = 0`. The tolerance `eta_k` and drift margin `r_drift` are set by the theory (Corollary 2 in the paper). The gate is the same for ALL drones — this is the key design choice that avoids mode mismatch across agents.

Local updates (T steps, no communication): Each drone takes projected gradient steps on either `f_i` (if `b_k = 1`, feasible round — improve coverage) or `g_i` (if `b_k = 0`, infeasible round — reduce energy usage). The constraint gradient being constant means infeasible-round updates simply push coverage times down proportionally to energy cost rates.

Output: Weighted average of all iterates from feasible rounds only.

Stepsize: Constant `gamma = D_x / (M * sqrt(K*T))` per Corollary 2. The key derived constants are:

- `L_G`: Lipschitz constant of the global constraint (from the `c_{ij}` values)
- `M_bar`: almost-sure bound on subgradient norms (worst case over both phases)
- `D_x`: prox diameter (bounded because X_i is bounded)
- `mu = 1` (Euclidean mirror map)

Condition (18): `T < mu * M^2 / (4 * delta * L_G * M_bar)` — limits inner steps to control drift. If violated, the gate becomes overly conservative.

Energy model and drone swapping

Energy draining is deterministic: drone `i` uses `sum_j c_{ij} * x_{ij}` Wh per round. Each drone tracks cumulative state-of-charge (SoC) across rounds, starting at `E_max`.

Swapping is an **external operational rule**, not driven by the algorithm. When a drone's SoC drops below `e_safe`, a fully charged standby replaces it. Since all drones are identical and `b` is fixed, the swap is transparent — the replacement inherits the iterate and problem data. The optimization doesn't know or care about swaps.

Track per-drone SoC for plotting (confirm no drone runs dry), but it does not affect the optimization logic.

What the experiments should demonstrate

The experiments validate **Theorem 1** from the paper:

1. **Feasibility guarantee:** Whenever the gate says feasible (`b_k = 1`), the constraint `G(x) ≤ eta_k` holds at every inner step of that round.
2. **O(1/sqrt(N)) convergence:** The objective gap `F(x_bar) - F(x*)` decreases at rate 1/sqrt(total inner updates).

3. **Communication–computation tradeoff:** Sweeping T (inner steps per round) shows that larger T = fewer communication rounds but a more conservative gate due to larger drift margin.

Tuning b matters: If the budget is too generous, the gate never flips to 0 (no interesting switching). If too tight, the algorithm mostly does constraint correction. Set b to roughly 60–80% of the "full demand" energy so early rounds see gate switching that transitions to mostly feasible as the algorithm converges.

Parameter summary

Quantity	Symbol	Role
Drones	n	5, 10, or 20
Sectors	d	4 or 8
Coverage demands	theta_{ij}	Minutes drone i should spend in sector j
Priority weights	alpha_{ij}	Importance of meeting demand in sector j
Energy cost rates	c_{ij}	Wh/min for drone i in sector j
Energy budget	b	Total Wh the fleet may spend per round
Round duration	T_max	Max minutes per drone per round
Outer rounds	K	200–1000
Inner steps	T	Sweep: 1, 5, 10, 25, 50
Confidence param	delta	0.1
Initial SoC	E_max	Per drone, e.g. 500 Wh
Swap threshold	e_safe	e.g. 50 Wh